

# Economic & Financial News Intelligence Platform

*Comparaison des Cahiers des Charges & Plan de Développement  
par Phases*

**Architecture Big Data | Pipeline Distribué | Médaille Bronze/Silver/  
Gold**

Python • Kafka • PySpark • PostgreSQL • MinIO • Docker • Streamlit

# 1. Comparaison des Deux Cahiers des Charges

Le tableau ci-dessous confronte le projet exemple fourni (PDF) avec votre cahier des charges (Economic & Financial News Intelligence Platform). Les différences en surbrillance verte constituent les apports et améliorations de votre projet.

| Critère            | Projet Exemple (PDF)                                 | Votre Projet (CdC)                                                       |
|--------------------|------------------------------------------------------|--------------------------------------------------------------------------|
| Domaine            | Actualité générale (tous sujets)                     | Actualité économique & financière                                        |
| Sources            | Hespress, Akhbarona, Al Jazeera, CNN, BBC, Reuters   | Reuters Business, Yahoo Finance, Hespress Économie                       |
| Champs collectés   | titre, auteur, date, catégorie, contenu, source, URL | Idem + langue, timestamp scraping, résumé, entities, sentiment           |
| Structure JSON     | Simple (champs de base)                              | Riche : metadata / article / entities / analytics / quality / governance |
| Ingestion          | Batch (toutes les heures) + Streaming                | Streaming principal via Kafka (JSON events)                              |
| Traitement         | Python ou SQL                                        | PySpark distribué (obligatoire)                                          |
| Stockage Data Lake | MinIO / HDFS / S3 (au choix)                         | MinIO ou stockage local (Docker)                                         |
| Data Warehouse     | Non précisé                                          | PostgreSQL (obligatoire)                                                 |
| Orchestration      | Apache Airflow (obligatoire)                         | Apache Airflow (optionnel)                                               |
| Visualisation      | Dashboard (outil libre)                              | Streamlit / Power BI / Superset                                          |

| Critère          | Projet Exemple (PDF)            | Votre Projet (CdC)                                              |
|------------------|---------------------------------|-----------------------------------------------------------------|
| Analyse NLP      | Non mentionné                   | Sentiment, score importance, fréquence mots-clés                |
| Entities         | Non mentionné                   | Pays, sociétés, devises, personnalités                          |
| Qualité données  | Complétude, cohérence, validité | Idem + duplicate_score, quality_score, missing_fields           |
| Gouvernance      | Documentation & traçabilité     | lineage_id, retention_policy, chemin stockage, pipeline_version |
| Conteneurisation | Non précisé                     | Docker (obligatoire)                                            |
| Langues          | Non précisé                     | Multi-langues : fr, ar, en (champ language dans JSON)           |

## 1.1 Points Communs

- Sources de données web → scraping automatique avec Python + BeautifulSoup/Scrapy
- Pipeline d'ingestion avec deux modes : batch et streaming
- Data Lake avec architecture Médaillon (Bronze / Silver / Gold)
- Transformations de données : nettoyage HTML, normalisation, validation
- Data Warehouse pour les données analytiques
- Dashboard de visualisation des tendances
- Contrôles de qualité des données et traçabilité (gouvernance)

## 1.2 Valeur Ajoutée de Votre Projet

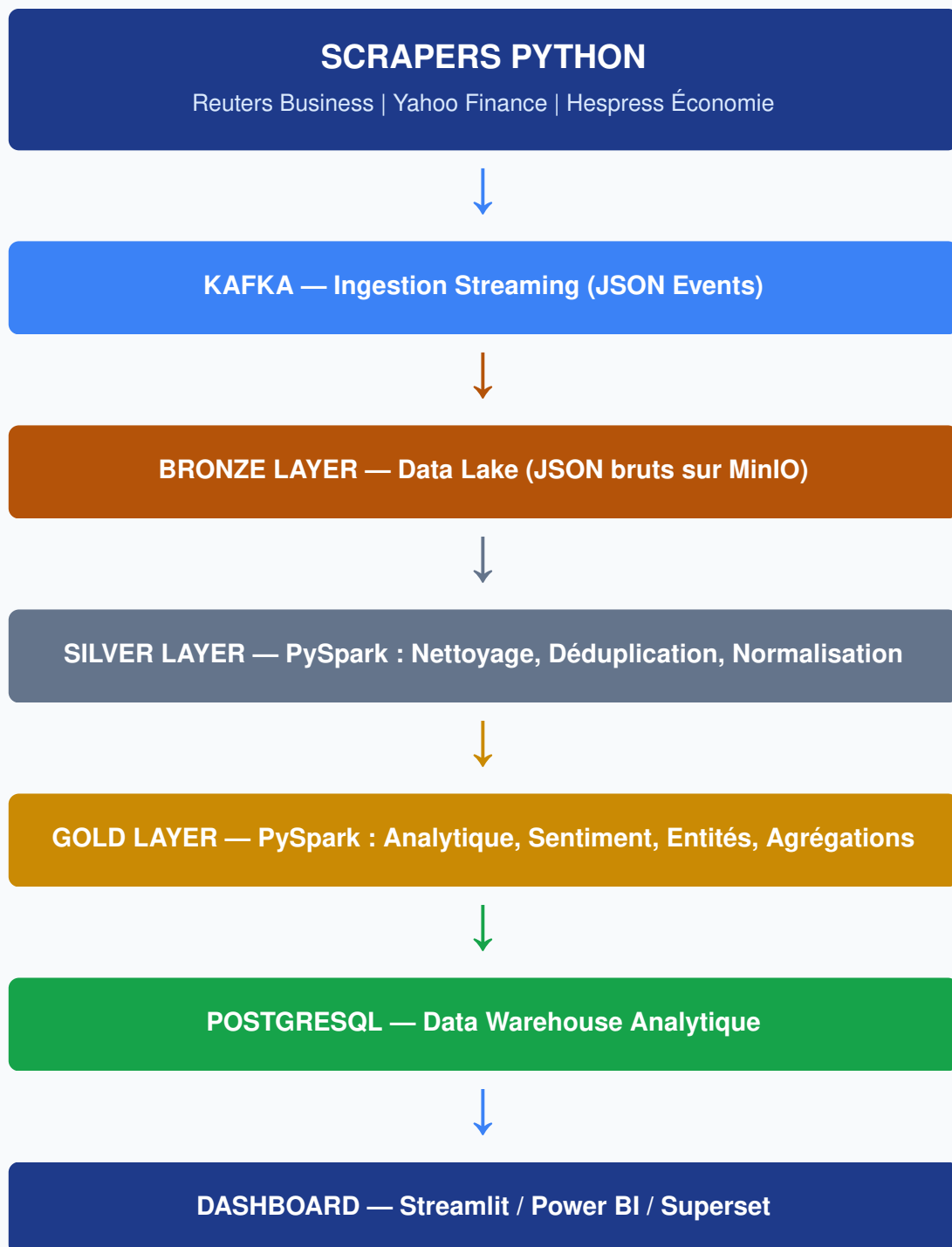
- Domaine ciblé : Actualité exclusivement économique & financière (marchés, crypto, énergie, immobilier...)
- JSON enrichi : 6 sections structurées (metadata, article, entities, analytics, quality, governance)
- Analyse NLP intégrée : sentiment analysis, score d'importance, fréquence des mots-clés
- Extraction d'entités : pays, entreprises, devises, personnalités mentionnées

- Conteneurisation Docker obligatoire : environnement reproductible et portable
- PostgreSQL défini comme Data Warehouse cible (pas laissé au choix)
- Support multi-langues : français, arabe, anglais (champ language dans chaque article)
- Scores de qualité quantifiés : duplicate\_score, quality\_score (valeurs numériques 0-1)
- Gouvernance avancée : lineage\_id, retention\_policy, pipeline\_version, collector\_node

## 2. Architecture Technique Globale

---

L'architecture suit un flux unidirectionnel depuis la collecte jusqu'à la visualisation, avec une couche de qualité et gouvernance transversale.



## 3. Découpage en Phases de Développement

Le projet est découpé en 9 phases progressives. Les phases 1 à 4 posent l'infrastructure et la collecte. Les phases 5 à 7 implémentent le traitement distribué. Les phases 8 et 9 livrent la valeur finale (dashboard + qualité/gouvernance).

### PHASE 1 Infrastructure & Environnement Docker

Objectif : Mettre en place l'environnement complet avant toute ligne de code métier. Tous les services doivent être opérationnels et communicants.

|                      |                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>      | Déployer tous les services via Docker Compose : Kafka, Zookeeper, MinIO, PostgreSQL, Spark. Aucun service installé manuellement.                                                                                                                                                                                                                                            |
| <b>Technologies</b>  | Docker, Docker Compose, Kafka 3.x, Zookeeper, MinIO, PostgreSQL 15, Apache Spark 3.x                                                                                                                                                                                                                                                                                        |
| <b>Démarche</b>      | <ol style="list-style-type: none"><li>1. Créer docker-compose.yml avec tous les services</li><li>2. Configurer les réseaux inter-services</li><li>3. Créer les volumes persistants</li><li>4. Vérifier la communication entre Kafka, Spark et MinIO</li><li>5. Initialiser les buckets MinIO (bronze, silver, gold)</li><li>6. Créer le schéma initial PostgreSQL</li></ol> |
| <b>Livrables</b>     | docker-compose.yml fonctionnel, services accessibles, schéma DB initialisé                                                                                                                                                                                                                                                                                                  |
| <b>Durée estimée</b> | 2-3 jours                                                                                                                                                                                                                                                                                                                                                                   |

Docker

Docker Compose

Kafka

MinIO

PostgreSQL

## PHASE 2 Collecte de Données — Scrapers Python

Objectif : Développer les scrapers pour les 3 sources définies dans le CdC. Chaque scraper produit un objet JSON structuré conforme au schéma défini.

|                       |                                                                                                                                                                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>       | Collecter les métadonnées et contenus. Construire le JSON enrichi.                                                                                                                                                                                                                             |
| <b>Sources cibles</b> | Reuters Business   Yahoo Finance News   Hespress Économie                                                                                                                                                                                                                                      |
| <b>Technologies</b>   | Python 3.10+, BeautifulSoup4, Scrapy (optionnel), requests, lxml, schedule                                                                                                                                                                                                                     |
| <b>Démarche</b>       | <ol style="list-style-type: none"> <li>1. Analyser la structure HTML</li> <li>2. Identifier les sélecteurs</li> <li>3. Développer un scraper par source</li> <li>4. Implémenter un scraper abstrait</li> <li>5. Générer l'article_id unique</li> <li>6. Ajouter gestion des erreurs</li> </ol> |
| <b>Livrables</b>      | 3 modules Python + base_scraper.py + tests unitaires                                                                                                                                                                                                                                           |
| <b>Durée estimée</b>  | 4-5 jours                                                                                                                                                                                                                                                                                      |

Python 3.10+

BeautifulSoup4

Scrapy

requests

## PHASE 3 Ingestion Kafka — Streaming des Articles

|                     |                                                                                                                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>     | Créer les Kafka Producers dans chaque scraper. Configurer les topics.                                                                                                                                                  |
| <b>Topics Kafka</b> | economic-news-raw   economic-news-errors                                                                                                                                                                               |
| <b>Démarche</b>     | <ol style="list-style-type: none"> <li>1. Créer la classe KafkaProducer avec sérialisation JSON</li> <li>2. Intégrer le producer dans chaque scraper</li> <li>3. Tester l'envoi avec kafka-console-consumer</li> </ol> |

|                      |                                                        |
|----------------------|--------------------------------------------------------|
| <b>Livrables</b>     | kafka_producer.py, topics configurés, consumer de test |
| <b>Durée estimée</b> | 2-3 jours                                              |

Kafka 3.x

kafka-python

JSON

## PHASE 4 Data Lake Bronze — Stockage des Données Brutes

|                        |                                                                                                                                                                                                           |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>        | Implémenter le Kafka Consumer qui écrit les articles bruts dans MinIO.                                                                                                                                    |
| <b>Chemin stockage</b> | /bronze/reuters/YYYY/MM/DD/HH/article_{id}.json                                                                                                                                                           |
| <b>Démarche</b>        | <ol style="list-style-type: none"><li>1. Créer le Consumer Kafka</li><li>2. Intégrer le client MinIO</li><li>3. Implémenter la logique de partitionnement</li><li>4. Écrire chaque message JSON</li></ol> |
| <b>Livrables</b>       | bronze_consumer.py, bucket MinIO bronze peuplé                                                                                                                                                            |
| <b>Durée estimée</b>   | 2 jours                                                                                                                                                                                                   |

MinIO

minio-py

kafka-python



## PHASE 5 Spark Processing — Couche Silver (Nettoyage & Validation)

|                        |                                                                                                                                                                                                                            |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>        | Développer le job Spark qui lit la couche Bronze, nettoie et normalise les données, puis écrit les résultats dans la couche Silver.                                                                                        |
| <b>Transformations</b> | <ol style="list-style-type: none"> <li>1. Nettoyage HTML</li> <li>2. Déduplication (hash du titre + source)</li> <li>3. Normalisation texte</li> <li>4. Parsing dates (ISO 8601)</li> <li>5. Validation de base</li> </ol> |
| <b>Démarche</b>        | <ol style="list-style-type: none"> <li>1. Lire JSON depuis Bronze</li> <li>2. Appliquer UDF et transformations DataFrame</li> <li>3. Écrire en Parquet dans Silver</li> </ol>                                              |
| <b>Livrables</b>       | spark_silver_job.py, udf_functions.py, Parquet en Silver                                                                                                                                                                   |
| <b>Durée estimée</b>   | 4-5 jours                                                                                                                                                                                                                  |

PySpark 3.x

Spark Streaming

dateparser

## PHASE 6 Spark Analytique — Couche Gold (Enrichissement)

|                    |                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>    | Calculer le sentiment, extraire les entités, produire les agrégations.                                |
| <b>Indicateurs</b> | Articles par source   Activité par heure   Mots-clés fréquents   Sentiment moyen   Score d'importance |

|                      |                                                                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Démarche</b>      | <ol style="list-style-type: none"> <li>1. Lire Parquet Silver</li> <li>2. Appliquer VADER (Sentiment)</li> <li>3. Extraire entités (spaCy)</li> <li>4. Créer tables analytiques et écrire en Gold</li> </ol> |
| <b>Livrables</b>     | spark_gold_job.py, nlp_utils.py, tables analytiques                                                                                                                                                          |
| <b>Durée estimée</b> | 5-6 jours                                                                                                                                                                                                    |

PySpark 3.x

NLTK/VADER

spaCy NER

Spark ML

## PHASE 7 Data Warehouse PostgreSQL — Chargement

|                      |                                                                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>      | Créer les tables analytiques dans PostgreSQL et charger les données Gold.                                                                                                      |
| <b>Schéma</b>        | articles   article_analytics   article_entities   daily_stats   keyword_trends                                                                                                 |
| <b>Démarche</b>      | <ol style="list-style-type: none"> <li>1. Migrations Alembic</li> <li>2. Configurer JDBC connector</li> <li>3. Implémenter l'upsert</li> <li>4. Créer index et vues</li> </ol> |
| <b>Livrables</b>     | Schema SQL complet, scripts de chargement, diagramme ERD                                                                                                                       |
| <b>Durée estimée</b> | 3-4 jours                                                                                                                                                                      |

PostgreSQL 15

SQLAlchemy

Spark JDBC

## PHASE 8 Dashboard & Visualisation

|                      |                                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Objectif</b>      | Créer le dashboard interactif qui visualise les données.                                                                                                            |
| <b>Vues</b>          | Globale   Flux Temps Réel   Top Sujets   Évolution Temporelle   Sentiments                                                                                          |
| <b>Démarche</b>      | <ol style="list-style-type: none"> <li>1. Créer app.py Streamlit</li> <li>2. Fonctions de requêtes PG</li> <li>3. Intégrer Plotly</li> <li>4. Dockeriser</li> </ol> |
| <b>Livrables</b>     | app.py Streamlit, dashboard accessible localement                                                                                                                   |
| <b>Durée estimée</b> | 4-5 jours                                                                                                                                                           |

Streamlit

Plotly

psycopg2

## PHASE 9 Qualité, Gouvernance & Orchestration

|                      |                                                                      |
|----------------------|----------------------------------------------------------------------|
| <b>Objectif</b>      | Formaliser les contrôles de qualité, mettre en place la traçabilité. |
| <b>Qualité</b>       | duplicate_score   quality_score   validations URL/date               |
| <b>Gouvernance</b>   | lineage_id   raw_storage_path   retention_policy                     |
| <b>Livrables</b>     | data_quality.py, logs JSON structurés, airflow_dag.py (opt.)         |
| <b>Durée estimée</b> | 3-4 jours                                                            |

Great Expectations

Airflow

JSON Logs

## 4. Résumé des Phases & Planning

| # | Phase                    | Technologies clés                   | Durée | Livrable principal         |
|---|--------------------------|-------------------------------------|-------|----------------------------|
| 1 | Infrastructure<br>Docker | Docker, Kafka, MinIO,<br>PostgreSQL | 2-3 j | docker-compose.yml         |
| 2 | Scrapers Python          | Python, BeautifulSoup,<br>Scrapy    | 4-5 j | 3 scrapers<br>fonctionnels |
| 3 | Ingestion Kafka          | kafka-python, JSON                  | 2-3 j | Topics Kafka<br>peuplés    |
| 4 | Bronze Layer             | MinIO SDK, Consumer Kafka           | 2 j   | Bucket bronze<br>peuplé    |
| 5 | Silver / Spark Clean     | PySpark, dateparser,<br>langdetect  | 4-5 j | Parquet Silver<br>propres  |
| 6 | Gold / Analytics         | PySpark, NLTK, spaCy,<br>Spark ML   | 5-6 j | Tables analytiques<br>Gold |
| 7 | Data Warehouse PG        | PostgreSQL, JDBC                    | 3-4 j | Schéma DB + vues           |
| 8 | Dashboard                | Streamlit, Plotly                   | 4-5 j | Dashboard interactif       |
| 9 | Qualité &<br>Gouvernance | Great Expectations, Airflow         | 3-4 j | Scores qualité + logs      |

**Durée totale estimée : 29 - 38 jours (binôme)**

Pour un binôme travaillant en parallèle, les phases 2-3 et 4-5 peuvent être réalisées simultanément, réduisant la durée effective à environ 18-25 jours.